PES UNIVERSITY, BANGALORE

Department of Computer Science and Engineering

Jackfruit Phase-1

Test Plan

for

ChatSphere – Real-Time Direct & Group Messaging Platform

Version 1.1

Prepared by Team 16 – Satvik L, Prithvi Aithal, Sourish Ganesh

PES University, Bangalore – Dept. of CSE, Section G

September 2026

Revision History

Name	Date	Reason For Changes	Version
Team 16	Sep 2026	Initial Test Plan baseline for Requirements submission	1.0
Team 16	Nov 2026	Test cases updated to reflect the changes made during Sprints 1-3 (Appendix A: TC-01 to TC-10 revised, TC-11 to TC-23 added); traceability (Appendix B) updated to the final test cases	1.1

Table of Contents

Revision History 2
Table of Contents 3
1. Test Plan Identifier 4
2. Introduction 4
2.1 Purpose 4
2.2 Scope 4
2.3 References 4
3. Test Items 4
4. Features to be Tested and Features Not to be Tested 5
4.1 Features to be Tested 5
4.2 Features Not to be Tested 5
5. Approach 5
5.1 Security Validation Approach 5
5.2 Functional Testing Approach 6
5.3 Non-Functional (Performance) Testing Approach 6
5.4 Regression and Unit Testing Approach 6
6. Item Pass/Fail Criteria 6
7. Suspension Criteria and Resumption Requirements 6
8. Test Deliverables 6
9. Testing Tasks 7
10. Environmental Needs 7
11. Responsibilities 7
12. Schedule 7
13. Risks and Contingencies 8
14. Approvals 8
Appendix A: Test Cases 9
Appendix B: Requirement Traceability Matrix 12

1. Test Plan Identifier

Identifier: TP-CHATSPHERE-16-1.1. This Test Plan corresponds to Version 1.1 of the ChatSphere Software Requirements Specification (SRS-CHATSPHERE-16-1.1) and to the Level 3 Software Engineering mini-project (UE24CS341A) Deliverable 2 (Test Plan) for Team 16.

2. Introduction

2.1 Purpose

This Test Plan defines the scope, approach, resources and schedule of the testing activities for ChatSphere. It follows the structure of the IEEE 829 Standard for Software Test Documentation, adapted for the course's stage-wise deliverable flow: at this stage the plan is planning-focused (test items, features, approach, criteria, schedule and a representative set of test cases); concrete pass/fail execution results will be recorded in the System Validation Report once Deliverables 3 and 4 (Design, Implementation & Testing) are complete.

2.2 Scope

Testing covers every functional requirement (FR-001 to FR-021), non-functional requirement (NFR-001, NFR-002) and security requirement (SEC-001 to SEC-006) defined in the SRS, across the five system features: Authentication & Presence Management, Direct Messaging, Group/Room Chat, Message History & Status Tracking, and Room Administration. Items explicitly out of scope for this phase (end-to-end encryption, voice/video calling, file/media sharing, multi-server scaling, third-party SSO) per Synopsis Section 7 are excluded from this Test Plan.

2.3 References

- ChatSphere Software Requirements Specification, Version 1.1 (SRS-CHATSPHERE-16-1.1).
- ChatSphere Synopsis / Project Proposal, Version 1.1.
- IEEE Std 829-2008, Standard for Software and System Test Documentation.
- PES University – “Jackfruit – Software Engineering Project Guidelines 2026” (UE24CS341A).

3. Test Items

The following software items, as they will exist at the end of each relevant sprint, are subject to this Test Plan:

Item	Description	Version / Sprint
Auth & Presence Service	Registration, login (JWT issuance), WebSocket handshake authentication, presence broadcast	Sprint 3, v1.0
Direct Messaging Service	1:1 message send/receive, typing indicator, delivery/read status	Sprint 4, v1.0
Room / Group Chat Service	Room create/join/leave, group message broadcast	Sprint 5, v1.0
Message History Module	Persistence and paginated retrieval of direct & group messages	Sprint 5, v1.0
Room Administration Module	Promote/remove member, admin-role enforcement (RBAC)	Sprint 5, v1.0
REST API layer	Express.js endpoints for auth and history retrieval	Sprint 3-5, v1.0

WebSocket Gateway	Socket.IO event routing for all real-time features	Sprint 3-5, v1.0

4. Features to be Tested and Features Not to be Tested

4.1 Features to be Tested

Every functional, non-functional and security requirement in SRS Sections 5 and 6 is in scope for testing, summarized by feature:

- Authentication & Presence — FR-001 to FR-006, SEC-001, SEC-004.
- Direct Messaging — FR-007 to FR-011, NFR-001.
- Group / Room Chat — FR-012 to FR-016, SEC-002.
- Message History & Status Tracking — FR-017, FR-018, NFR-002.
- Room Administration — FR-019 to FR-021, SEC-002.
- Cross-cutting security — SEC-001 to SEC-006 (see Section 5.1).

4.2 Features Not to be Tested

The following are out of scope for this phase, per SRS Section 2.6 and Synopsis Section 7, and are therefore not tested:

- End-to-end message encryption.
- Voice / video calling.
- File, image or media sharing.
- Multi-instance / horizontal scaling of the Socket.IO layer.
- Third-party SSO (Google/GitHub login).
- Native mobile push notifications.

5. Approach

Testing follows a layered strategy: unit tests for isolated business logic, integration tests for service-to-service and service-to-database behaviour, functional (black-box) tests derived directly from each FR, and dedicated security and performance testing passes. Each test case in Appendix A is written against a specific requirement ID so results feed directly into the Requirement Traceability Matrix (Appendix B).

5.1 Security Validation Approach

Security validation is treated as a first-class, dedicated testing activity rather than a by-product of functional testing, in line with SEC-OBJ-1 to SEC-OBJ-4 in the SRS. It comprises:

- Authentication/authorization testing: verifying that REST endpoints and WebSocket connections reject missing, expired or tampered JWTs (SEC-004), and that room-administration actions are rejected for any user without the admin role for that room (SEC-002, FR-021).
- Negative/penetration-style testing: manually attempting room-admin REST/WebSocket calls with a non-admin JWT, an expired JWT and a JWT with a modified payload/signature; attempting to access a room's messages as a non-member (FR-015).
- Input-safety testing: submitting message content containing HTML/script payloads and confirming the content is sanitized/escaped before storage and before rendering, ruling out stored XSS (SEC-003).

- Configuration/code review: confirming, by inspection, that passwords are hashed with bcrypt and never persisted in plaintext (SEC-001) and that security-relevant events (failed logins, unauthorized admin attempts) are logged (SEC-006).
- Abuse/rate-limit testing: sending message-send events at a high rate from a single connection and confirming the rate limiter engages (SEC-005).

Security test results, including the penetration-style checks, will be consolidated into the System Validation Report at Deliverable 5, per SRS Section 8.

5.2 Functional Testing Approach

Black-box functional tests are derived one-to-one from the functional requirements in SRS Section 5. Each feature owner (per Synopsis Section 9) writes and executes the functional tests for their owned feature during the sprint in which it is built, with peer review of test cases before merge.

5.3 Non-Functional (Performance) Testing Approach

Latency and throughput targets (NFR-001, NFR-002) are validated with a scripted load-test client that opens multiple concurrent WebSocket connections and REST sessions, measuring the 95th-percentile message delivery time and REST response time against the defined test load.

5.4 Regression and Unit Testing Approach

Unit tests (Jest) cover business logic in each service (auth, messaging, room, presence) in isolation, using mocked sockets and a mocked/in-memory MongoDB instance. The full unit and integration suite is re-run automatically on every pull request via the GitHub Actions CI/CD pipeline, acting as the regression safety net for later sprints.

6. Item Pass/Fail Criteria

A test item (feature or requirement) passes when: (a) every test case tagged to it in Appendix A executes with the expected result; (b) no High-priority defect remains open against it; and (c) for NFR-001/NFR-002, the measured 95th-percentile value meets the stated threshold under the defined test load. A test case itself passes only when the actual result exactly matches its expected result; any deviation is logged as a defect and the test case is marked Failed until re-verified.

7. Suspension Criteria and Resumption Requirements

- Testing of a feature is suspended if a blocking defect prevents further test cases for that feature from executing (e.g. the server fails to start, or the database connection cannot be established).
- Testing is suspended feature-wide if a High-severity security defect is found (e.g. a non-admin action is not rejected), until a fix is merged.
- Testing resumes once the blocking defect is fixed, the CI pipeline is green, and the affected test cases are re-run from the point of suspension.

8. Test Deliverables

- This Test Plan document (TP-CHATSPHERE-16-1.1).
- Test case specifications (Appendix A) and the Requirement Traceability Matrix (Appendix B).
- Automated test suite (Jest/Supertest) and CI run logs.
- Postman collection and results for REST API testing.

- Final System Validation Report (Deliverable 5), consolidating pass/fail results, defect log, and security/penetration-test findings.

9. Testing Tasks

Task	Description	Owner
Write test cases	Derive test cases from each FR/NFR/SEC requirement	Feature owner
Set up test environment	Docker-composed Node.js, MongoDB, Redis test stack; CI runner config	Sourish Ganesh
Implement unit/integration tests	Jest/Supertest suites per module	Feature owner
Execute functional tests	Run black-box tests against running server	All members
Execute security tests	Run authentication/authorization/negative/XSS/rate-limit checks	Sourish Ganesh
Execute performance tests	Run scripted load test against NFR-001/002	All members
Log and triage defects	Record, prioritize and assign defects found during testing	All members
Consolidate results	Produce the System Validation Report	All members

10. Environmental Needs

- Hardware/OS: Linux-based Docker host (development laptops + GitHub Actions runners).
- Software: Node.js 20 LTS, MongoDB 6+, Redis 7+, Jest, Supertest, Postman.
- Network: local Docker network for integration tests; HTTPS/WSS simulated with self-signed certificates in the test environment.
- Test data: a seeded set of test user accounts, rooms and messages created by a setup script before each test run and torn down after.

11. Responsibilities

Member	Testing Responsibility
Sourish Ganesh	Authentication & Presence tests; all security/penetration-style tests; CI/CD pipeline and test-environment setup
Prithvi Aithal	Direct Messaging and Message History tests; API/Postman test collections
Satvik L	Group/Room Chat and Room Administration tests; frontend-facing usability checks
All members	Unit tests for own owned feature; peer review of test cases; defect triage

12. Schedule

Testing is embedded in the six-sprint schedule defined in SRS Section 9.2, rather than run as a single end-of-project phase:

Sprint	Testing Activity
Sprint 1-2	Test Plan drafted; test environment and CI pipeline stood up

Sprint 3	Unit + functional tests for Authentication & Presence
Sprint 4	Unit + functional tests for Direct Messaging; usability walkthrough
Sprint 5	Unit + functional tests for Room Chat, History and Room Administration; full security and performance test pass
Sprint 6	Regression run, defect closure, System Validation Report

13. Risks and Contingencies

Risk	Contingency
A security defect (e.g. missing RBAC check) is found late in Sprint 5	Security tests for each module are run incrementally from Sprint 3 onward, not deferred to Sprint 5, to surface issues early
Load-test environment cannot reach the concurrency needed to validate NFR-001	Fall back to a scaled-down concurrency target with the shortfall documented in the System Validation Report
A team member's owned feature is delayed, compressing its test window	Cross-train a second member per module (per Synopsis Section 9) so testing can proceed in parallel

14. Approvals

Name	Role	Signature / Date
Satvik L	Team Member	
Prithvi Aithal	Team Member	
Sourish Ganesh	Team Member	
Course Faculty	Evaluator	

Appendix A: Test Cases

The following ten test cases give initial, representative coverage of functional, non-functional and security requirements. Additional test cases will be added as design and implementation proceed (Deliverables 3 and 4).

TC ID	Requirement(s)	Description	Preconditions	Steps	Expected Result
TC-01	FR-001	Register with unique username and email	Application reachable; username/email not already registered	1. Submit registration form with unique username, email, valid password.	HTTP 201 with the new userId; the stored password is a bcrypt hash, never the plaintext (SEC-001).
TC-02	FR-001 (negative)	Register with a duplicate email	An account already exists with the given email	1. Submit registration form re-using an existing email.	HTTP 400 VALIDATION_FAILED with a clear message; no duplicate account is created (email and username are unique ignoring case).
TC-03	FR-003	Login with valid credentials	A registered account exists	1. Submit login form with correct username/email and password.	HTTP 200 with a signed JWT (claims sub and username); the client opens an authenticated Socket.IO session with it.
TC-04	FR-004, SEC-004	WebSocket handshake and REST call with an invalid JWT	A JWT has expired, or is signed with another key, or has been tampered with	1. Attempt a Socket.IO connection with the token. 2. Call GET /api/messages/:id with it.	Socket connection refused with connect_error AUTH_INVALID_TOKEN; REST returns 401 AUTH_INVALID_TOKEN; no session is established.
TC-05	FR-008, NFR-001	Direct message delivered to online recipient	Sender and recipient are both authenticated and connected	1. Sender emits message:send { toUserId, content }. 2. Measure time until message:new arrives on the recipient.	Recipient receives message:new in real time; sender gets message:status "sent" and an ack; under test load 95% of messages arrive within 300 ms.
TC-06	FR-009	Direct message to offline recipient	Recipient is authenticated but not currently connected	1. Sender sends a direct message. 2. Recipient connects afterward.	Message is stored with status "sent"; on the recipient's next connection it is pushed as message:new and the sender receives message:status "delivered".
TC-07	FR-012, FR-014	Create room and broadcast group message	User is authenticated	1. room:create with a unique name. 2. A second user sends room:join. 3. Creator	Room is created with the creator as admin; members get room:memberJoined; the message is

				sends message:send { roomId, content }.	broadcast as message:new to every connected member.
TC-08	FR-021, SEC-002	Non-admin attempts to remove a room member	A room exists with a non-admin member and at least one other member	1. Non-admin member issues a remove-member action.	The action is rejected with AUTH_FORBIDDEN (socket error event / ack); the target member is not removed; the attempt is security-logged (SEC-006).
TC-09	SEC-003	Message content sanitized against stored XSS	Sender is authenticated and a member of the target room	1. Send a message containing an embedded script tag.	Content is stored HTML-escaped (e.g. <script>) and rendered as plain text; no markup or script is created in any client.
TC-10	FR-018	Retrieve paginated message history	A conversation/room with more messages than one page exists	1. Request message history for the conversation/room with a page size parameter.	HTTP 200 { messages[], nextPage }: newest first, page size respected, nextPage null on the last page; a non-participant gets 403 AUTH_FORBIDDEN.
TC-11	FR-003, SEC-006	Login with wrong password or unknown user	A registered account exists	1. Log in with a wrong password. 2. Log in with an unknown username.	Both rejected with HTTP 401 and the same message (no account enumeration); each attempt is written to the security log.
TC-12	FR-011	Message status: sent, delivered, read	Sender and recipient connected	1. Sender sends a DM. 2. Recipient opens the conversation.	Sender receives sent, then delivered, then read (message:status); the status never moves backwards.
TC-13	FR-010	Typing indicator	A DM conversation exists; both users connected	1. User A types in the composer. 2. A stops typing for 3 s.	B sees "A is typing..." and it clears after A stops; a non-participant receives no typing:update.
TC-14	FR-017, SAD 3.4.2	Retried send is not duplicated	Sender connected	1. Send a DM with idempotency key K. 2. Send it again with the same key K.	Exactly one message is stored and delivered; the retry is acknowledged with the original message.
TC-15	SEC-005, SEC-006	Rate limit on message sending	Sender connected	1. Send messages faster than the limit (burst above 10).	Sends over the limit are rejected with RATE_LIMITED and are neither stored nor broadcast; the burst is security-logged.

TC-16	FR-001 (negative)	Username that differs only in case	Account "alice" exists	1. Register "ALICE" with another email. 2. Register "Bob" and "bob" at the same time.	Rejected with VALIDATION_FAILED; exactly one account exists per username.
TC-17	FR-013	Join and leave a room	A group room exists; the user is not a member	1. room:join. 2. room:leave.	Members are notified (room:memberJoined / room:memberLeft); after leaving, the user receives no more room messages.
TC-18	FR-015	Non-member cannot read or post in a room	A group room exists; the user is not a member	1. Send a message to the room. 2. Request the room's history.	Both rejected with AUTH_FORBIDDEN; nothing is stored.
TC-19	FR-005, FR-006	Presence: online and offline	Users A and B share a conversation	1. B connects. 2. B disconnects.	A receives presence:update online when B connects and offline within 5 s of B disconnecting.
TC-20	FR-019, FR-020	Admin promotes and removes members	A room with an admin and two members	1. Admin promotes member 1. 2. Admin removes member 2.	Member 1 becomes admin and member 2 is removed; all members are notified.
TC-21	FR-016	A room always keeps at least one admin	A room with a single admin	1. The only admin tries to leave the room.	Rejected with VALIDATION_FAILED; the room still has an admin.
TC-22	NFR-002	REST response time under load	Server running with seeded conversations	1. Issue 500 authenticated history requests, 25 at a time.	95% of responses complete within 1 s, with no errors.
TC-23	FR-004, SEC-004	Token of a user that no longer exists	A user logged in and the account was then deleted	1. Call a REST endpoint with the old token. 2. Open a Socket.IO connection with it.	Both rejected with AUTH_INVALID_TOKEN.

Appendix B: Requirement Traceability Matrix

This matrix maps each SRS requirement to the test case(s) that verify it, or to the planned validation approach for requirements not yet covered by a listed test case (verified instead by code/configuration review, or by a test case to be added in a later sprint per SRS Section 8).

Requirement ID	Brief Description	Test Case(s) / Validation
FR-001	Unique user registration	TC-01, TC-02, TC-16
FR-002	Passwords hashed with bcrypt	TC-01 (stored hash verified by an automated test); code review (see SEC-001)
FR-003	Login issues signed JWT	TC-03, TC-11
FR-004	WebSocket handshake JWT check	TC-04, TC-23
FR-005	Broadcast online presence	TC-19
FR-006	Broadcast offline presence within 5s	TC-19
FR-007	Send direct message	TC-05
FR-008	Real-time delivery when online	TC-05
FR-009	Queue & deliver to offline recipient	TC-06
FR-010	Typing indicator	TC-13
FR-011	Message status sent/delivered/read	TC-12
FR-012	Create room, creator is first admin	TC-07
FR-013	Join / leave room	TC-17
FR-014	Broadcast group message	TC-07
FR-015	Only members can view/send in room	TC-18, TC-10
FR-016	Room always retains ≥ 1 admin	TC-21
FR-017	Persist every message	TC-05, TC-06, TC-14
FR-018	Paginated history retrieval	TC-10
FR-019	Admin promotes a member	TC-20
FR-020	Admin removes a member	TC-20
FR-021	Reject non-admin admin actions	TC-08
NFR-001	95% message delivery ≤ 300 ms	TC-05 (load test)
NFR-002	95% REST response ≤ 1 s	TC-22 (load test)
SEC-001	No plaintext passwords	TC-01 (automated bcrypt-hash check); code review
SEC-002	RBAC on room-admin actions	TC-08
SEC-003	Sanitize message content (XSS)	TC-09
SEC-004	Reject expired/invalid JWTs	TC-04, TC-23
SEC-005	Rate-limit message-send events	TC-15
SEC-006	Log security-relevant events	TC-11, TC-15, TC-08 (security log entries)